Recent Developments in Algorithm Design

Lecture 2: Generalizations and Variants of Set Cover (Hellerstein)



Recall Linear Program Duality

Linear Program (for minimization problem) in canonical form

- minimize $\sum_{j=1}^{n} c_j x_j$

such that $\sum_{j=1}^{n} a_{i,j} x_j \geq b_i$ for $i, \ldots, m$

$x_j \geq 0$ for $j = 1, \ldots, n$

Linear Program (for minimization problem) in canonical form — vector notation version

- minimize $c^T x$

  such that $Ax \leq b$

  $x \geq 0$

Linear Program in canonical form and its dual — vector notation version

- minimize $c^T x$

  such that $Ax \geq b$

  $x \geq 0$

- maximize $b^T y$

  such that $A^T y \leq c$

  $y \geq 0$

Linear Program (for minimization problem) in canonical form, and its dual

- minimize $\sum_{j=1}^{n} c_j x_j$

  such that $\sum_{j=1}^{n} a_{i,j} x_j \geq b_i$ for $i, \ldots, m$

  $x_j \geq 0$ for $j = 1, \ldots, n$

- maximize $\sum_{i=1}^{m} b_i y_i$

  such that $\sum_{i=1}^{m} a_{i,j} y_i \leq c_j$ for $j = 1, \ldots, n$

  $y_i \geq 0$ for $i = 1, \ldots, m$

Terminology

- A solution to an LP is “feasible” if it satisfies all the constraints
- Given a feasible solution to an LP, we say that a constraint of the LP is made “tight” by this solution if it causes the left hand side of the constraint to equal the right hand side

Weak duality

- If $\hat{x}$ is a feasible solution to the primal LP (i.e., it satisfies all the constraints of the primal LP), and $\hat{y}$ is a feasible solution to the dual LP (i.e., it satisfies all the constraints of the dual LP), then

$$c^T \hat{x} \geq b^T \hat{y}$$

Strong duality

- If x^* is an optimal solution to the primal LP and y^* is an optimal solution to the dual LP, then

$$c^T x^* = b^T y^*$$

Primal-Dual Algorithm for Set Cover

Recall Set Cover Problem

- Input: Ground set (universe) $\mathcal{U} = \{e_1, \ldots, e_n\}$ and family of subsets $\mathcal{F} = \{S_1, \ldots, S_m\}$ where each $S_i \subseteq \mathcal{U}$, such that $\bigcup_{i=1}^{m} S_i = \mathcal{U}$

- Task: Find a minimum size subset of $\mathcal{F}$ that covers all the elements of $\mathcal{U}$

LP for Set Cover

- minimize $\sum_{S \in \mathcal{F}} x_S$

  such that $\sum_{S : e \in S} x_S \geq 1 \qquad \forall e \in \mathcal{U}$

  $x_S \geq 0 \qquad \forall S \in \mathcal{F}$

LP for weighted Set Cover

- Each $S \in \mathcal{F}$ has an associated weight $w_S \geq 0$.
  Want minimum weight set cover.
- minimize $\sum_{S \in \mathcal{F}} w_S x_S$

  such that $\sum_{S : e \in S} x_S \geq 1 \qquad \forall e \in \mathcal{U}$

  $x_S \geq 0 \qquad \forall S \in \mathcal{F}$

Primal LP and Dual LP for Set Cover

- Primal LP: minimize $\sum_{S \in \mathcal{F}} w_S x_S$

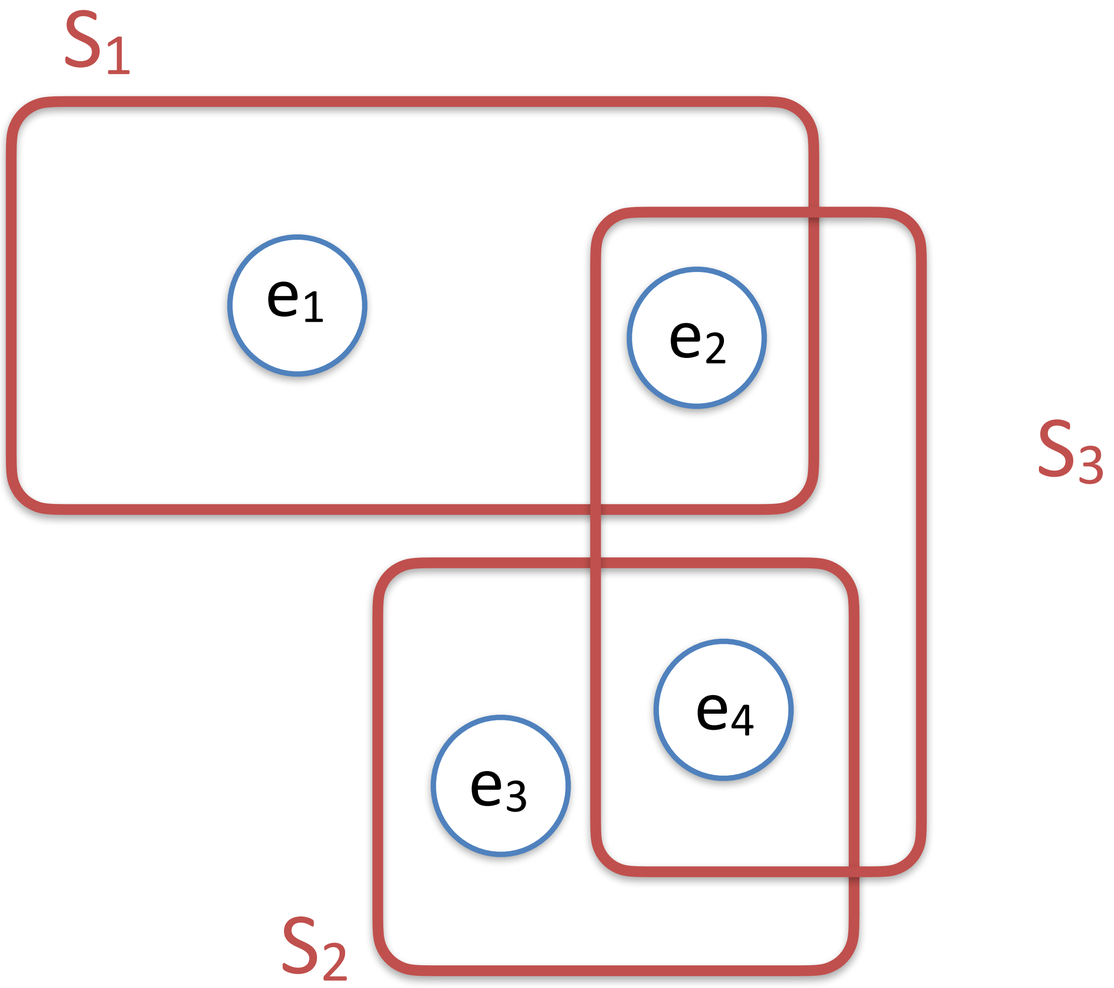
  such that $\sum_{S : e \in S} x_S \geq 1 \qquad \forall e \in \mathcal{U}$

  $x_S \geq 0 \qquad \forall S \in \mathcal{F}$

- Dual LP: maximize $\sum_{e \in \mathcal{U}} y_e$

  such that $\sum_{e : e \in S} y_e \leq w_S \qquad \forall S \in \mathcal{F}$

  $y_e \geq 0 \qquad \forall e \in \mathcal{U}$



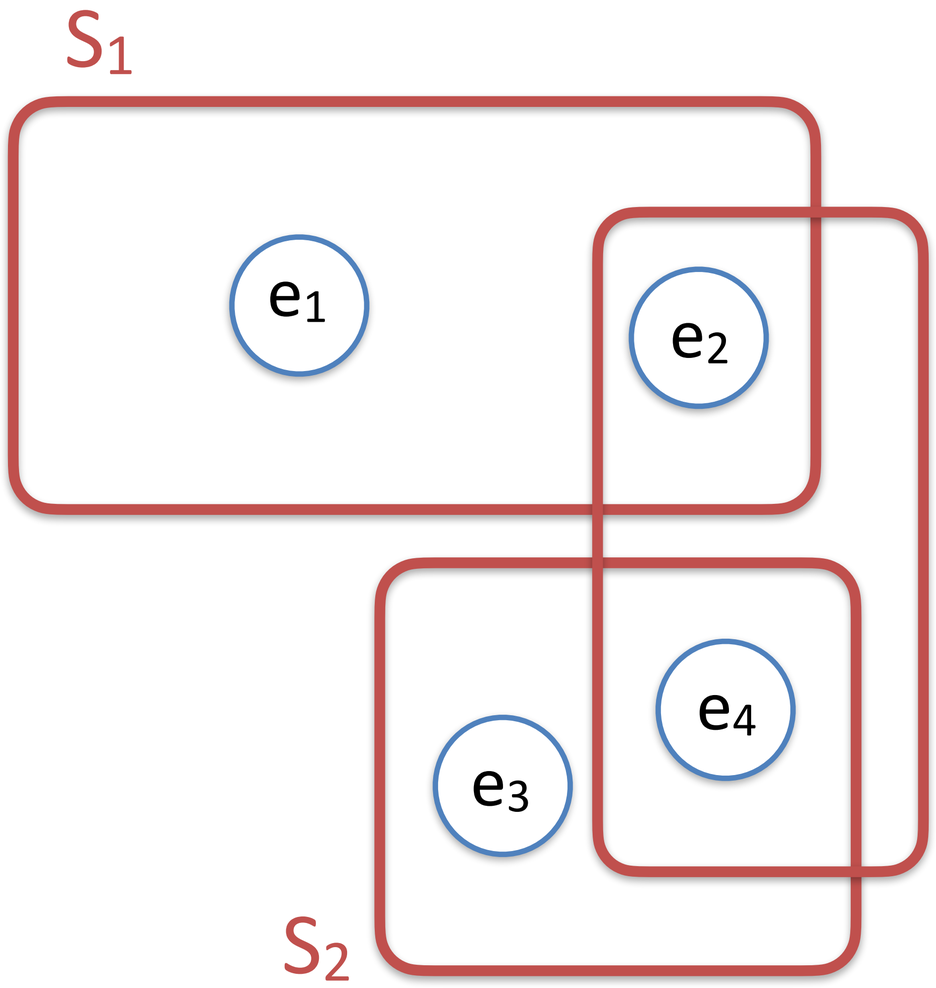
Primal:

$$\textbf{min } 5x_{S_1} + 7x_{S_2} + 6x_{S_3}$$

Dual:

$$\textbf{max } y_{e_1} + y_{e_2} + y_{e_3} + y_{e_4}$$

$$w(S_1) = 5, \quad w(S_2) = 7, \quad w(S_3) = 6$$



$$w(S_1) = 5, \quad w(S_2) = 7, \quad w(S_3) = 6$$

Primal:

$$\textbf{min } 5x_{S_1} + 7x_{S_2} + 6x_{S_3}$$

s.t.

$$x_{S_1} \geq 1$$
$$x_{S_1} + x_{S_3} \geq 1$$
$$x_{S_2} \geq 1$$
$$x_{S_2} + x_{S_3} \geq 1$$
$$x_{S_1}, x_{S_3}, x_{S_3} \geq 0$$

Dual:

$$\textbf{max } y_{e_1} + y_{e_2} + y_{e_3} + y_{e_4}$$

s.t.

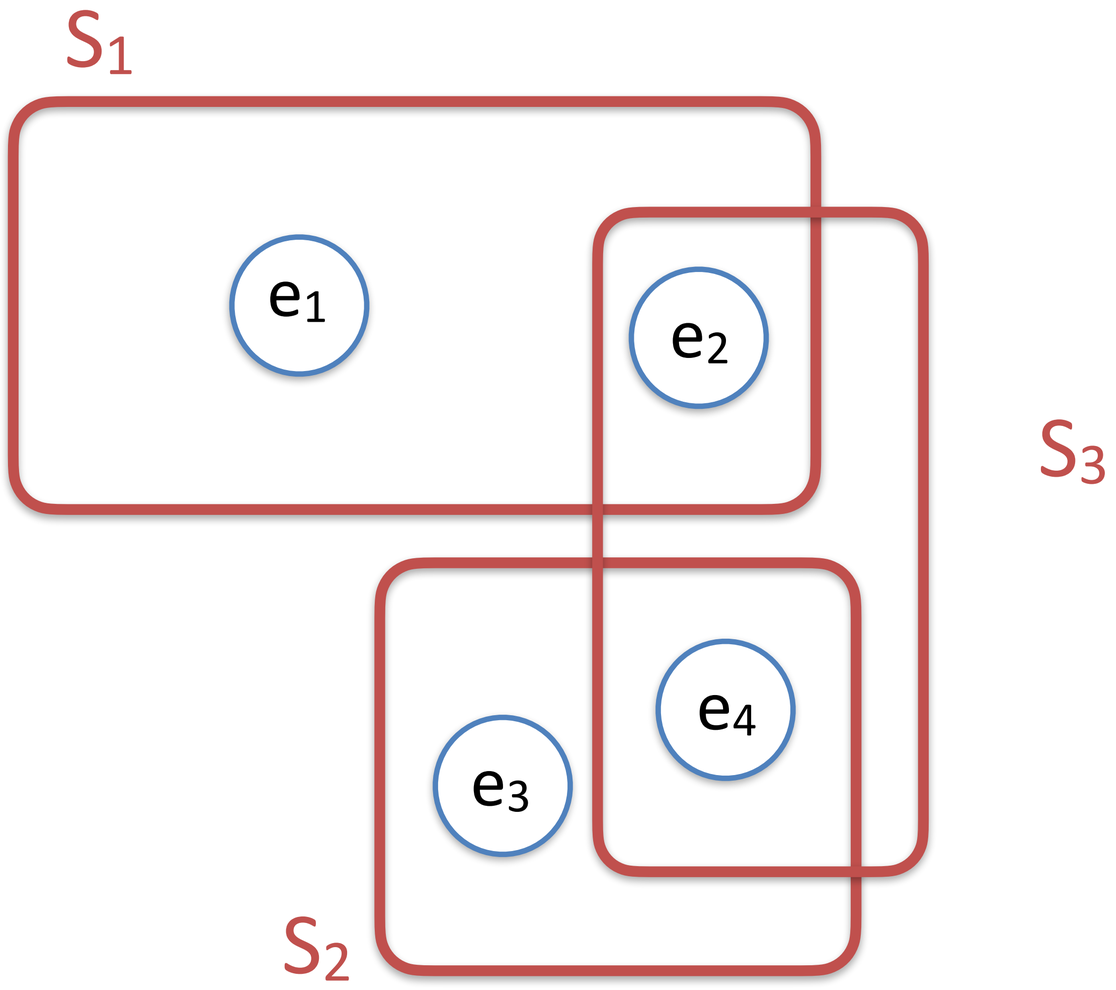
$$y_{e_1} + y_{e_2} \leq 5$$
$$y_{e_3} + y_{e_4} \leq 7$$
$$y_{e_2} + y_{e_4} \leq 6$$
$$y_{e_1}, y_{e_2}, y_{e_3}, y_{e_4} \geq 0$$

Primal-Dual algorithm for set cover

- Another approximation algorithm for (weighted) Set Cover, produces a cover of total weight $f * OPT$, where f is max number of sets containing an element $e \in \mathcal{U}$.
- Simple, combinatorial algorithm, doesn't require using an LP solver to produce an optimal solution to the linear progam
- But LP and its dual used in motivation behind algorithm and in its analysis

Primal Dual algorithm for set cover

- Begin with feasible solution $\hat{y} = 0$ for the Dual LP. (i.e., for all e, $\hat{y}_e$=0 is value assigned to y_e)
- $F' = \emptyset$. \\ We will add subsets $S \in \mathcal{F}$ to F' until subsets in F' cover all the elements in $\mathcal{U}$.
- while there is an element $e \in \mathcal{U}$ such that e not covered by F'
  - Increase $\hat{y}_e$ until some constraint in the dual that contains y_e becomes tight
    \\ if a constraint containing $\hat{y}_e$ already tight, “increase” $\hat{y}_e$ by 0
  - Let S be subset associated with this constraint
  - $F' = F' \cup \{S\}$
- return F'



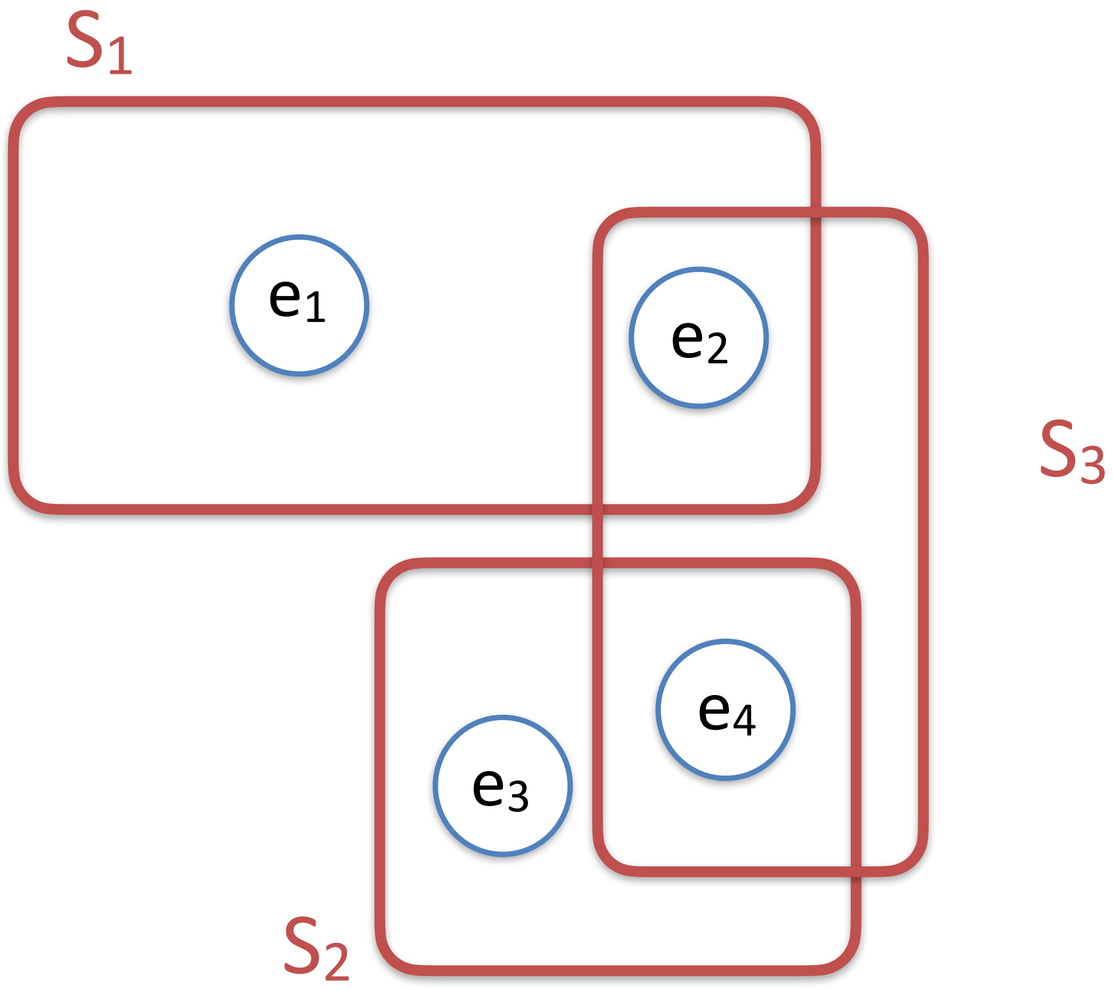
$$w(S_1) = 5, \quad w(S_2) = 7, \quad w(S_3) = 6$$

Dual:

$$\textbf{max } y_{e_1} + y_{e_2} + y_{e_3} + y_{e_4}$$

s.t.

$$y_{e_1} + y_{e_2} \leq 5$$
$$y_{e_3} + y_{e_4} \leq 7$$
$$y_{e_2} + y_{e_4} \leq 6$$
$$y_{e_1}, y_{e_2}, y_{e_3}, y_{e_4} \geq 0$$



$$w(S_1) = 5, \quad w(S_2) = 7, \quad w(S_3) = 6$$

$$\hat{y}_{e_1} = \hat{y}_{e_2} = \hat{y}_{e_3} = \hat{y}_{e_4} = 0$$

Increase $\hat{y}_{e_2}$

$$\hat{y}_{e_2} = 5$$

Add S_1 **to cover**

e_3 **and** e_4 **still uncovered**

Increase y_{e_4}

$$\hat{y}_{e_4} = 2$$

Add S_2 **to cover**

Dual:

$$\textbf{max } y_{e_1} + y_{e_2} + y_{e_3} + y_{e_4}$$

s.t.

$$y_{e_1} + y_{e_2} \leq 5$$
$$y_{e_3} + y_{e_4} \leq 7$$
$$y_{e_2} + y_{e_4} \leq 6$$

$$y_{e_1}, y_{e_2}, y_{e_3}, y_{e_4} \geq 0$$

Analysis of Primal-Dual algorithm for Set Cover

- Thm: Primal-Dual algorithm constructs a set cover of size at most $f \times OPT$, where OPT is the value of the optimal solution (minimum weight set cover), and f is the max number of sets $S \in \mathcal{F}$ covering any element $e \in \mathcal{U}$
- Pf:

  Can show that algorithm successfully produces cover F'
- Consider cover F' constructed by the algorithm, and final value of $\hat{y}$. Total weight of constructed cover F' is $\sum_{S \in F'} w_S$

  For each $S \in F'$, the corresponding dual constraint is tight, $\sum_{e \in S} \hat{y}_e = w_S$, so the total weight of F' is

$$\sum_{S \in F'} w_S = \sum_{S \in F'} \sum_{e \in S} \hat{y}_e$$

$$\leq f \sum_{e \in \mathcal{U}} \hat{y}_e \quad \text{since each } e \in \mathcal{U} \text{ belongs to at most } f \text{ sets } S \in \mathcal{F}$$

$$\leq f \times LP \quad \text{where LP is the optimal value of the LP, by duality and fact that } \hat{y} \text{ is feasible solution to Dual LP}$$

$$\leq f \times OPT$$

Greedy Algorithm for Weighted Set Cover

Generalization of (primal) Greedy algorithm for weighted set cover

- Previous greedy algorithm didn't handle weights on the subsets, just needed to minimize size of cover
- Natural generalization of greedy algorithm, using greedy rule:
  - Choose subset S maximizing

$$\frac{\text{number uncovered elements in S}}{w_S}$$

    maximizing “bang-for-the-buck”

- Same approximation factor as for unweighted greedy algorithm, total weight of constructed cover is at most $\ln(|\mathcal{U}|+1)OPT$

Summary of Algorithms for Weighted Set Cover

Summary of presented Weighted Set-Cover Algorithms

	Approximation factor
Primal Dual	f
Greedy	ln U + 1
Solve LP and round deterministically	f
Solve LP, randomized rounding with alteration	ln U + 1 (expected)

Submodular (Set) Cover

Submodular (Set) Cover

- Let V be a finite set. Call the elements of V "items"

- Let $u : 2^V \to \mathbb{R}^{\geq 0}$ Function u assigns non-negative real value to each subset of items

  We will call u a "utility function"

- Monotonicity:

  Say u is monotone if for all A, B where $A \subseteq B \subseteq V$, $u(A) \leq u(B)$

  Adding additional items to a set can never decrease utility

Submodularity

- Definition:
  A function $u : 2^V \rightarrow \mathscr{R}^{\geq 0}$ is said to be *submodular* if for all A, B such that $A \subseteq B \subseteq V$, and $i \in V \backslash B$,

$$u(A \cup \{i\}) - u(A) \geq u(B \cup \{i\}) - u(B)$$

- Submodularity is sometimes called the *diminishing returns property.* Why?

Equivalent Definition of Submodularity

- Definition:
  A function $u : 2^V \rightarrow \mathscr{R}^{\geq 0}$ is said to be *submodular* if for all A, B such that $A, B \subseteq V$, and $i \in V \backslash B$,

$$u(A \cup B) + u(A \cap B) \leq u(A) + u(B)$$

Coverage functions

- Recall that an instance of the set cover problem is a set system consisting of universe $\mathcal{U}$ and a family $\mathcal{F}$ of subsets of $\mathcal{U}$
- Consider the utility function $u : 2^V \rightarrow \mathcal{R}^{\geq 0}$ where $V = \mathcal{F}$ and for all $F \subseteq \mathcal{F}$

  $u(F) = |\bigcup_{S \in F} S| =$ # elements of $\mathcal{U}$ covered by the sets S in F
- This type of utility function, based on a set system, is called a “coverage function”
- The “items” of the coverage function correspond to the subsets S in $\mathcal{F}$ (each item is a subset of elements of universe $\mathcal{U}$)
- If u is a coverage function, it is
  - monotone
  - submodular
  - has the property that $u(\emptyset) = 0$

Other examples of submodular functions

- Rank function of a vector space
  - V = set of all vectors in a vector space, and $u(V') =$ rank of space spanned by vectors in V'
- Modular (additive) functions
  - Each item $i \in V$ has an associated real weight w_i and $u(V') = \sum_{i \in V} w_i$

    If weights are non-negative, then u is monotone.
- “Budget additive” functions
  - Each item $i \in V$ has an associated weight $w_i \geq 0$ and for some $B \geq 0$, $u(V') = \min\{B, \sum_{i \in V} w_i\}$
- Non-monotone example: Graph cut functions
  - $V =$ set of all vertices in a graph G, $u(V')$ = number of edges (u, v) of G with $u \in V'$, $v \in V \backslash V'$

Generalization of set cover: Submodular Cover

- Problem: Given a finite set V, a utility function $u : 2^V \to \mathbb{Z}^{\geq 0}$ such that u is monotone, submodular, and satisfies $u(\emptyset) = 0$, find a minimum size subset $V' \subseteq V$ such that $u(V') = u(V)$
- Notes:
  - Problem statement doesn't mention how function u is given as input. We assume that it is given by an oracle that, given as input a subset $V' \subseteq V$, will return $u(V')$ in constant time.
  - Call $V' \subseteq V$ such that $u(V') = u(V)$ a "cover" of u
  - Set Cover is the special case of this problem where u is a coverage function (where the answers to the oracle queries are easily computed from the given set system)
  - Can extend to real-valued utility functions but results get a little messier

Generalization of Max (Set) Coverage Problem: Submodular Max Coverage Problem

- Problem: Given a finite set V, a utility function $u : 2^V \to \mathbb{R}^{\geq 0}$ such that u is monotone, submodular, and satisfies $u(\emptyset) = 0$, and an integer $k \geq 0$ such that $k \leq |V|$, find a subset $V' \subseteq V$ of size k that maximizes $u(V')$

- Greedy algorithm and its analysis almost same as they were for Max (Set) Coverage.

Greedy algorithm for Submodular Max Coverage problem

- Input: Finite set V of items, oracle for utility function $u : 2^V \rightarrow \mathscr{R}^{\geq 0}$, such that u is monotone, submodular, and $u(\emptyset) = 0$
- $V'_0 = \emptyset$ \\ V'_t will contain the first t items chosen
- for $t = 1$ to k
  - using oracle for u, find an item $i \in V \backslash V'_{t-1}$ that maximizes $u(V'_{t-1} \cup \{i\}) - u(V'_{t-1})$
    call this item $i*$ \\ $i*$ is the greedy choice
  - $V'_t = V'_{t-1} \cup \{i*\}$
- return V'_k

Analysis of Greedy Algorithm for Submodular Max Coverage problem

- Let V^* be an optimal solution, a set of k items that maximizes $u(V^*)$. Let $OPT = u(V^*)$
- At start of t-th greedy step, think of $OPT - u(V_{t-1})$ as the remaining distance to OPT
- In t-th greedy step, greedily choose item to add to V'_{t-1}
- If you added all the elements in V^* to V'_{t-1}, you'd reach utility value OPT, so increase in utility would be $OPT - u(V'_{t-1})$.
  - By submodularity and monotonicity of u, can show there must be an element of V^* that when added to V'_{t-1}would increase utility by at least $\frac{1}{|V^*|} = \frac{1}{k}$ of the remaining distance to OPT $\Rightarrow$

    after adding t-th greedy element, remaining distance to goal, $OPT - u(V_t) \leq (1 - \frac{1}{|OPT|}) * (OPT - u(V_{t-1}))$
- Inductively: $OPT - u(V'_k) \leq (1 - \frac{1}{k})^k OPT \leq \frac{1}{e} OPT \Rightarrow u(V'_k) \geq (1 - \frac{1}{e}) OPT$
- Therefore, Greedy outputs a solution that achieves utility at least $(1 - \frac{1}{e}) OPT$

Greedy algorithm for Submodular Cover

- Input: Finite set V of items, oracle for utility function $u : 2^V \rightarrow \mathscr{R}^{\geq 0}$, such that u is monotone, submodular, and $u(\emptyset) = 0$
- t=0
- $V'_t = \emptyset$ \\ V'_t contains items already put into V' at end of step t of greedy algorithm
- while $u(V'_t) \neq u(V)$
  - $t = t + 1$
  - using oracle for u, find an item $i \in V \backslash V'_{t-1}$ that maximizes $u(V'_{t-1} \cup \{i\}) - u(V'_{t-1})$
    call this item i^* \\ i^* is the greedy choice
  - $V'_t = V'_{t-1} \cup \{i^*\}$
- return V'_t

Analysis of Greedy Algorithm for Submodular Cover

- Let OPT be the minimum size of a cover of u, i.e., min size of a subset V^* such that $u(V^*) = u(V)$
- At start of t-th greedy step, think of $u(V) - u(V_{t-1})$ as the remaining distance to the goal utility value, $u(V)$
- As in analysis of max-coverage can show that inductively,
  $u(V) - u(V'_t) \leq (1 - \frac{1}{OPT})^t u(V) \ < e^{-\frac{t}{OPT}} u(V)$
- Setting $t = \lceil OPT \ln u(V) \rceil$, we get $u(V) - u(V'_t) < 1$, which implies that $u(V'_t) = u(V)$, since u is an integer valued function
- Therefore, Greedy algorithm runs for at most $\lceil OPT \ln u(V) \rceil \leq OPT(\ln u(V) + 1)$ steps, so its output solution has size at most $OPT(\ln u(V) + 1)$

Sequencing problems

Sequencing problems

- Problems where we want to find the optimal order in which to do something
- Examples:
  - Traveling salesman problem
    - Find the “best” permutation of vertices, given graph with weighted edges
    - NP-hard problem
    - polytime 1.5-approximation algorithm (assuming weights obey triangle inequality)

- Scheduling problems, e.g.. Min-sum completion time problem:
  - Set of n “jobs” to be scheduled on a single “processor”
  - Processor can only process one job at a time
  - Given the length ℓ_j of job j for j=1,…,n
  - Find permutation of jobs that minimizes the sum of completion times of the jobs (same as minimizing the average completion time)
  - Optimal solution is to order jobs in increasing length
  - If jobs have weights w_j , and want to minimize weighted sum of completion times
    - exercise: determine optimal solution for this case

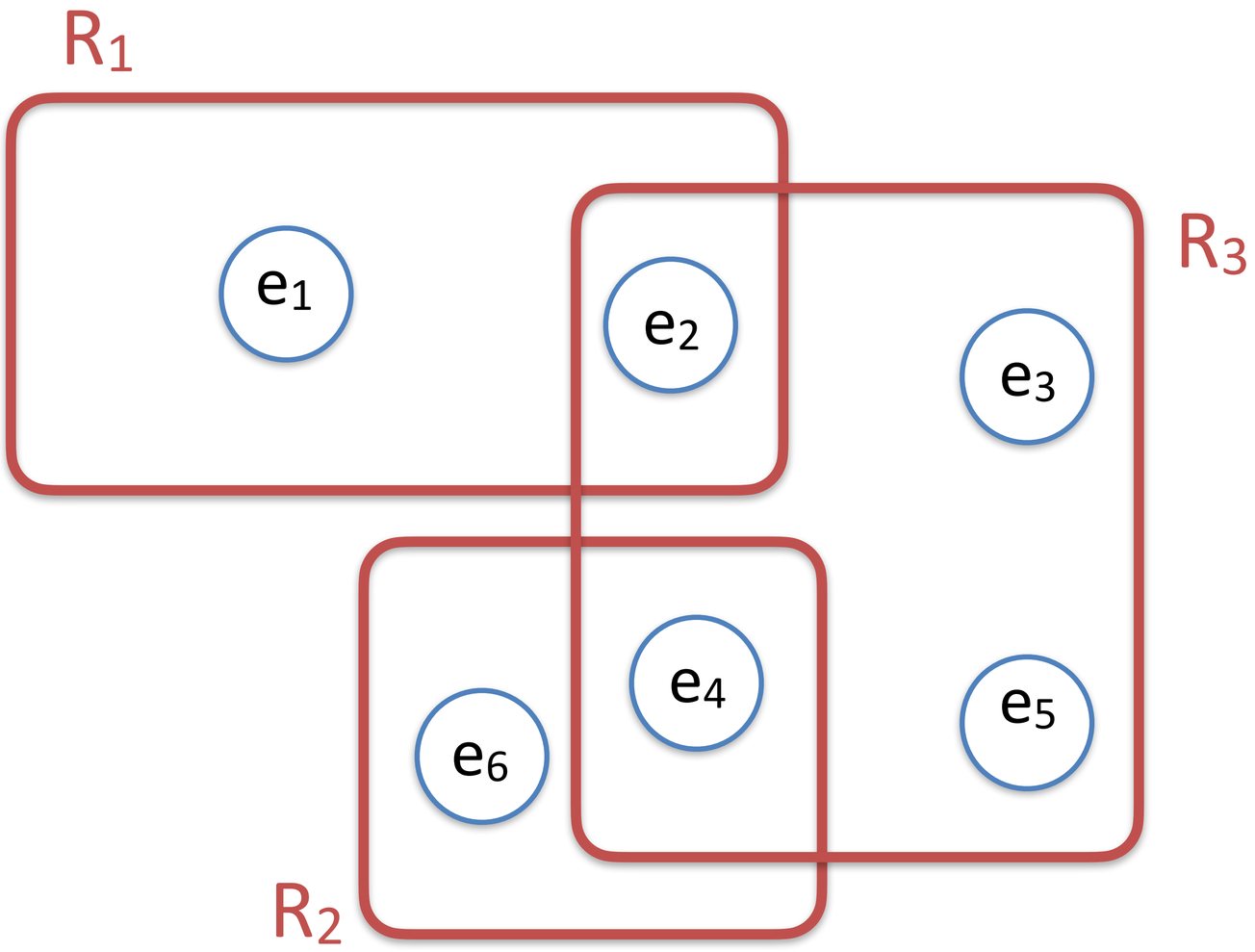
Min-Sum Set Cover

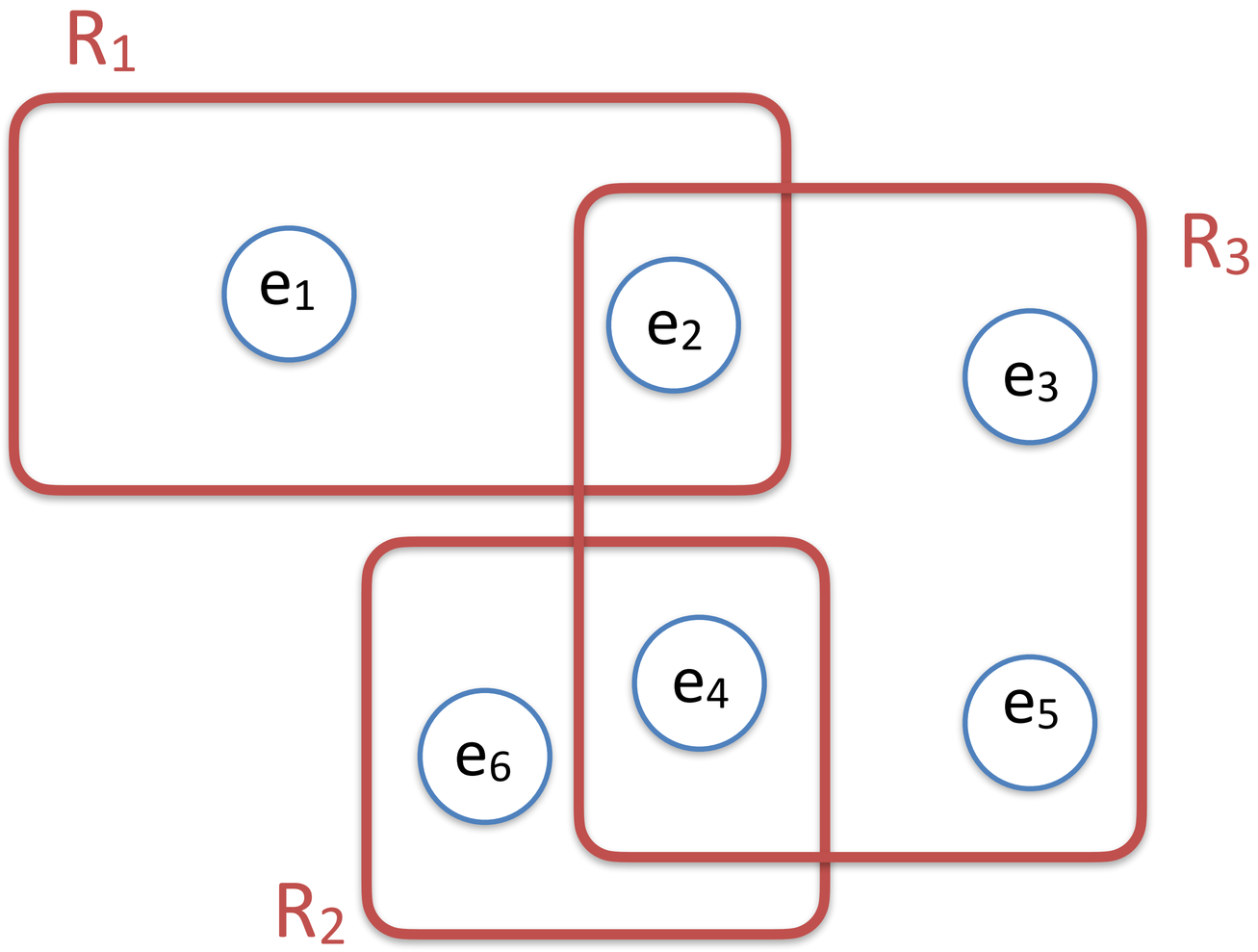
Min-Sum Set Cover [Lovasz et al. 02]

- Input: Ground set (universe) $\mathcal{U} = \{e_1, \ldots, e_m\}$ and family of subsets $\mathcal{F} = \{R_1, \ldots, R_n\}$ where each $R_i \subseteq \mathcal{U}$, such that $\bigcup_{i=1}^{n} R_i = \mathcal{U}$

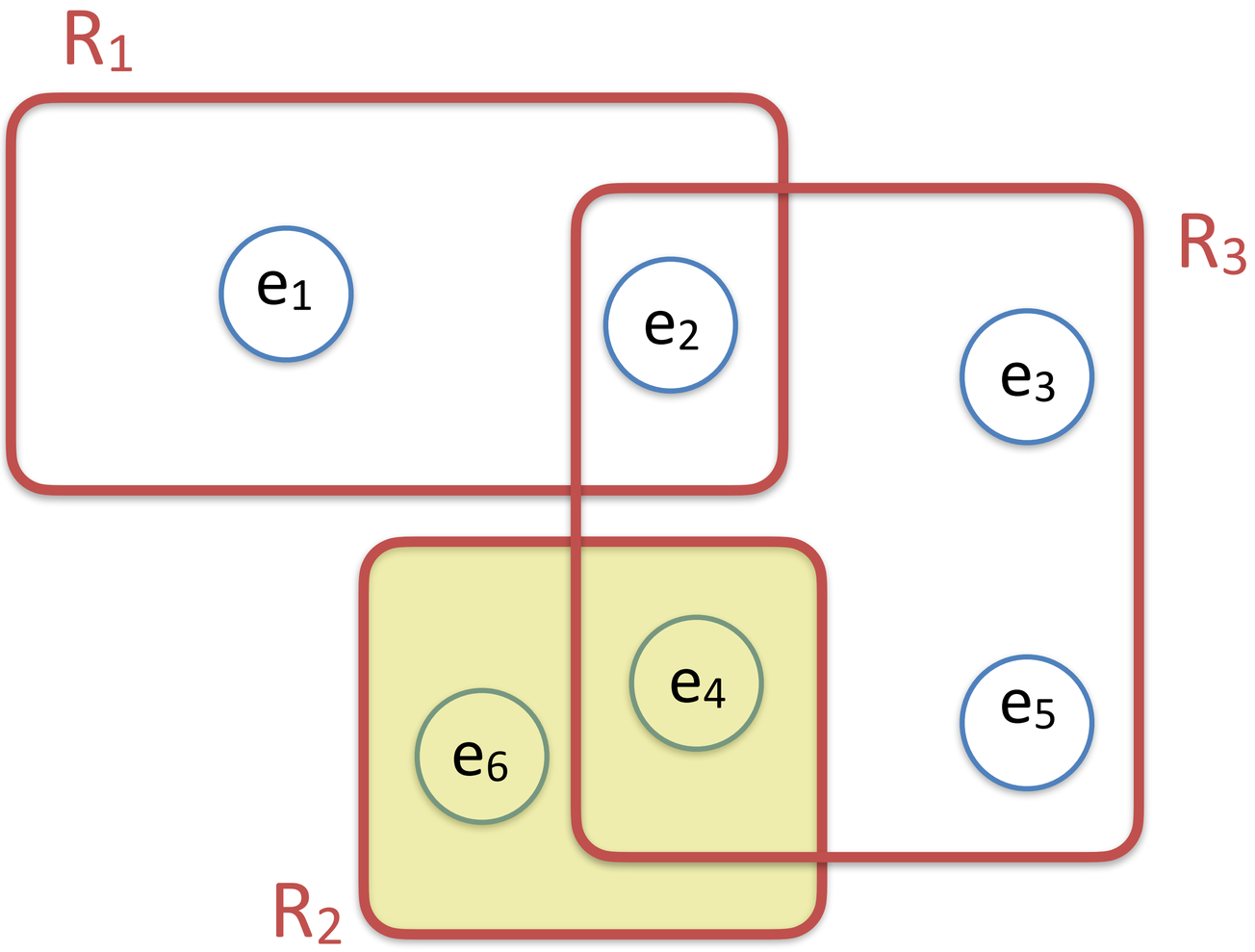
- Task: Find the permutation of the subsets that minimizes the sum of the *covering times* of the ground elements

  - If an element is covered by the jth element in the permutation, we say that it is covered at time j

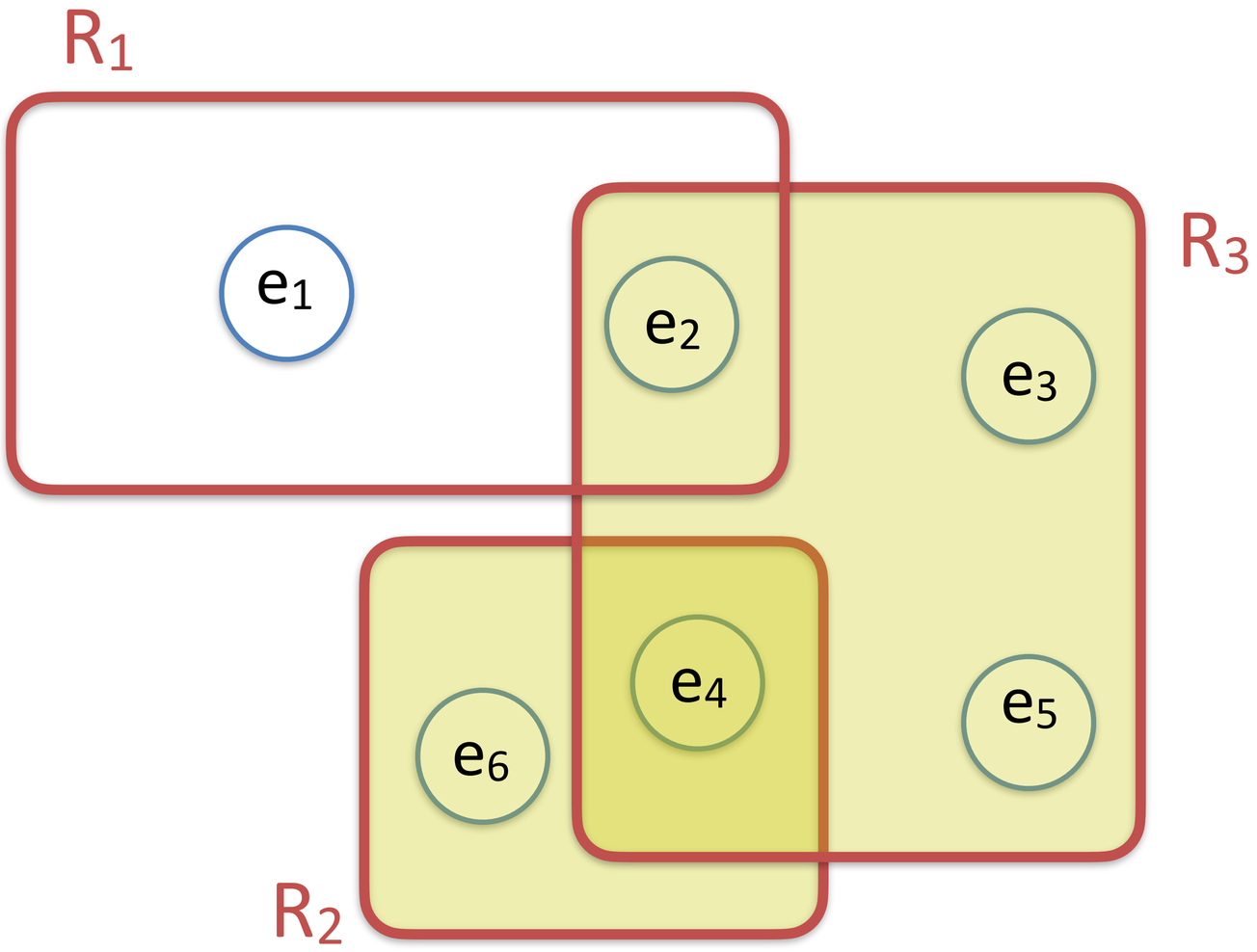

R1
R3
R2
e1
e2
e3
e4
e5
e6



R_2, R_3, R_1

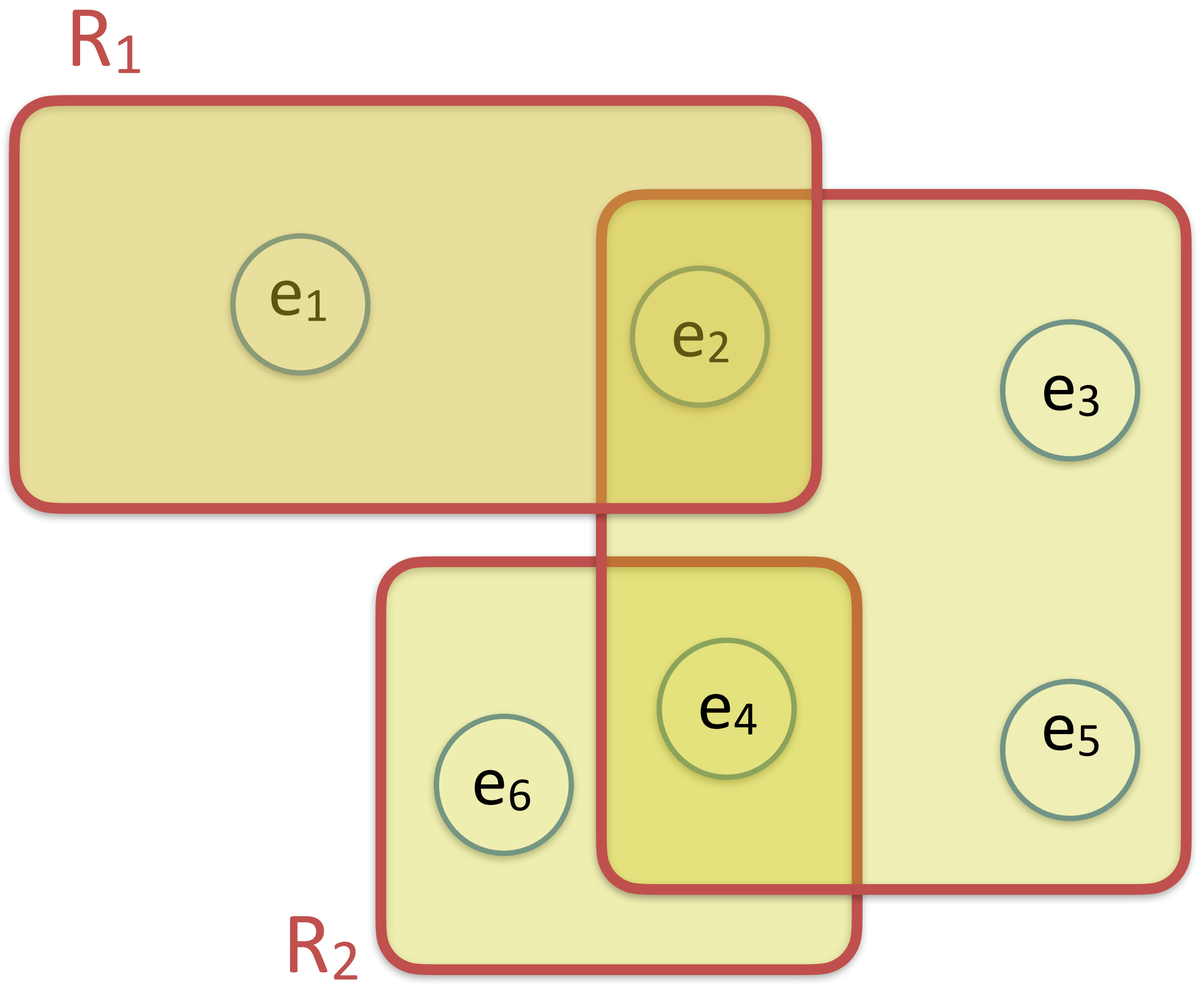


R_2, R_3, R_1 covers e_4, e_6 with R_2 (at time 1)



R_2, R_3, R_1 covers e_4, e_6 with R_2 (at time 1)

then e_2, e_3, e_5 with R_3 (at time 2)



Sum of cover times is
2*1+3*2+1*3 = 11

R_2, R_3, R_1 covers e_4, e_6 with R_2 (at time 1)

then e_2, e_3, e_5 with R_3 (at time 2)

then e_1 with R_1 (at time 3)

Greedy Algorithm

- Greedy Rule: Choose subset that covers the maximum number of uncovered elements
- Same greedy algorithm we used for “classical” set cover problem.
- What approximation factor does it achieve for Min-Sum Set Cover problem?

Approximation factor for Greedy Algorithm applied to Min-Sum Set Cover?

- Consider “bad” instance for greedy algorithm applied to Classical Set Cover. Greedy algorithm doesn’t do so badly!

Greedy Algorithm

- Thm [Lovasz et al. 02] :
  Greedy Algorithm is a 4-approximation algorithm for Min-Sum Set Cover

Proof based on histograms

Reformulation of Min-Sum Set Cover Problem in terms of "utility"

- Let $I = \{1, \ldots, m\}$
- Utility: Define $u : 2^I \rightarrow \mathscr{R}^{\geq 0}$ s.t. for $I' \subseteq I$,

  $u(I') = |\bigcup_{k \in I'} S_k|$

  = # elts of $\mathscr{U}$ covered by subset S_k such that $k \in I'$
- For permutation π of I, $i \geq 0$, define π^i = set of first i items in π
- Min-Sum Set Cover: Find permutation π of elements of N minimizing

$$\sum_{i=1}^{m} i \times (u(\pi^i) - u(\pi^{i-1}))$$

Greedy Algorithm

- Greedy Rule: If I' is the set of $i \in I$ already chosen for permutation, next add the item k maximizing increase in utility

$$u(I' \cup \{k\}) - u(I')$$

- Thm: Greedy Algorithm is a 4-approximation algorithm for Min-Sum Set Cover Problem

Proof based on histograms

Taxicab story

- Group of friends who get in taxicab together, not all going to same location
- Taxi goes to desired locations in some order, dropping off some passengers at each location
- Suppose each passenger has to pay for for travel before they are dropped off (!) at a rate of $1 per stop, e.g., if passenger is dropped off at the 3rd stop, their fare is $3.
- Driver could have each passenger pay their total fare when dropped off. So at stop i, total amount paid to the driver is

  $i \times$ (number of people dropped off at stop i)
- Alternatively, driver could have the passengers pay as they go: each time the taxi stops, everyone in the taxi has to give the driver $1. So at stop i, amount paid to driver is

  1 x number of people still in taxi when arrive at stop i
- Either way, driver earns the same amount of money for trip!

Pf of Theorem:

Given a permutation π of the elements in $I = \{1,2,\ldots,m\}$ and a prefix π^k of π, consisting of the first k sets in π,

let $r(\pi^k) = \sum_{i=1}^{k} i \times (u(\pi^i) - u(\pi^{i-1}))$

= sum of covering times of the first k items of π

- We're seeking a permutation π of the elements in $I = \{1, \ldots, m\}$ that minimizing the sum of the covering times, $r(\pi)$
- Total utility: $n = u(I)$ [number of elements in universe]

  Increase in utility at step i: $u(\pi^i) - u(\pi^{i-1})$
- Analogy to taxi: elements in $\mathcal{U}$ are passengers, each time step is a stop of taxi, being covered at step i is leaving taxi at stop i
- Pay when utility increases

$$r(\pi) = \sum_{i=1}^{m} i \times (u(\pi^i) - u(\pi^{i-1}))$$

- Pay as you go

$$r(\pi) = \sum_{i=1}^{m} 1 \times (u(I) - u(\pi^{i-1}))$$

Let G be permutation produced by running the greedy algorithm.

Let T be the permutation minimizing $r()$, i.e., T is an optimal solution to our Min-Sum problem We will prove

Claim: $r(G) \leq 4r(T)$

Which implies G is a 4-approx solution to our Min-Sum problem, proving the theorem.

- Think of each permutation as adding subsets, step by step
- In each step, cover more elements of $\mathcal{U}$, utility increases
- At start, $u(G^0) = 0$ and at end, $u(G^m) = u(G) = u(I)$
- View G as a journey from utility 0 to utility $u(I)$ [NOT as a journey through time!]
- In ith step of G, utility increases from $u(G^{i-1})$ to $u(G^i)$
- and similarly for optimal permutation T
- We will compare utility at start of ith step of G to utility at end of jth step of T

- We will use the following lemma.

  Lemma:

  If $\dfrac{u(G^i) - u(G^{i-1})}{u(N) - u(G^{i-1})} < \dfrac{1}{2j}$ then $u(N) - u(T^j) \geq \dfrac{1}{2}(u(I) - u(G^{i-1}))$

$u(G^0)$ $u(I)$

- Lemma: If $\frac{u(G^i) - u(G^{i-1})}{u(N) - u(G^{i-1})} < \frac{1}{2j}$ then $u(I) - u(T^j) \geq \frac{1}{2}(u(I) - u(G^{i-1}))$
- Pf: Assume $\frac{u(G^i) - u(G^{i-1})}{u(I) - u(G^{i-1})} < \frac{1}{2j}$. By Greedy Rule, if add single item to G^{i-1}, can't increase utility by more than $u(G^i) - u(G^{i-1})$.
- If add all j items in T^j to G^{i-1}, can't increase utility by more than $j * (u(G^i) - u(G^{i-1}))$

  $u(G^{i-1} \cup T^j) - u(G^{i-1}) \leq j * (u(G^i) - u(G^{i-1}))$

  $\Rightarrow u(T^j) - u(G^{i-1}) \leq j * (u(G^i) - u(G^{i-1})$ because $u(T^j) \leq u(T^j \cup G^{i-1})$

  $\leq \frac{1}{2}(u(I) - u(G^{i-1}))$ by assumption

  $\Rightarrow u(I) - u(T^j) \geq \frac{1}{2}(u(I) - u(G^{j-1}))$

Claim: $r(G) \leq 4r(T)$

Pf of claim:

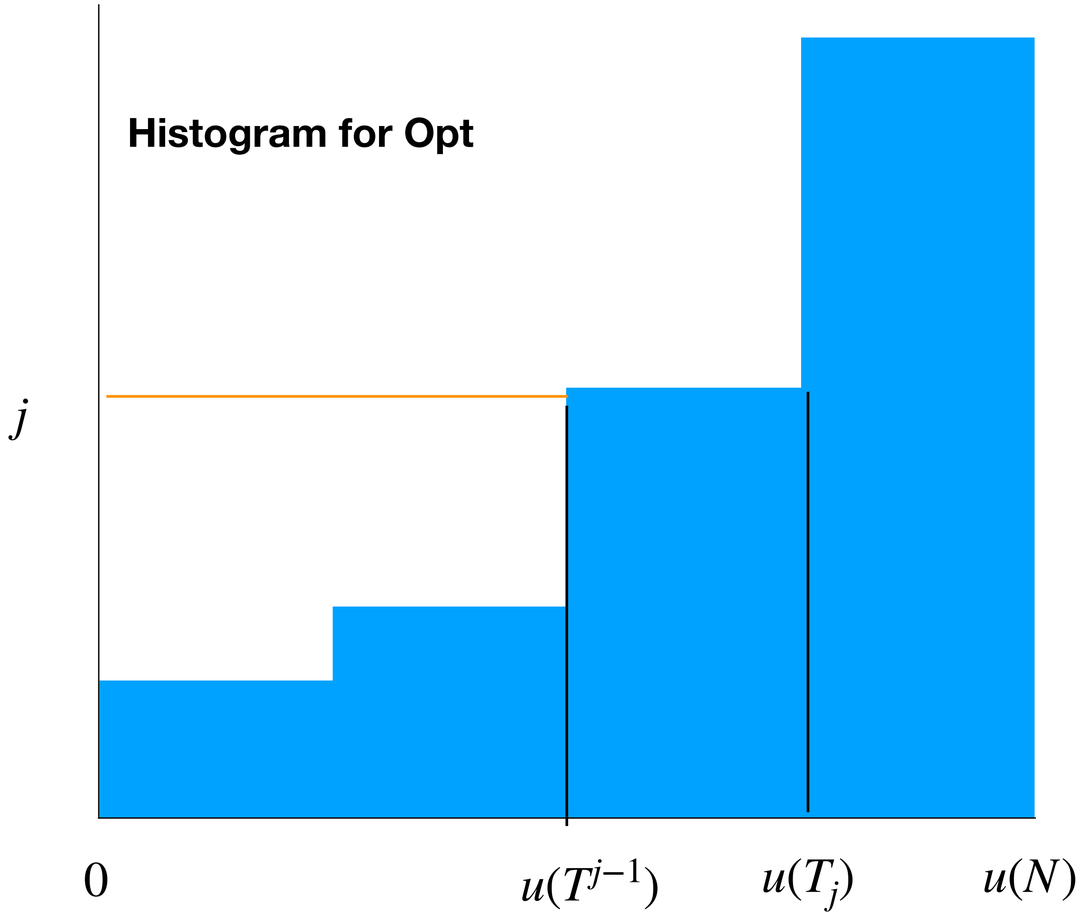
Consider a histogram for T.

Horizontal axis: labeled from 0 to $u(I)$

Bar corresponding to T_j goes from

$$u(T^{j-1}) \text{ to } u(T_j)$$

of height j



Area under histogram is $r(T) = \sum_j j \times (u(T^j) - u(T^{j-1}))$

Claim: $r(G) \leq 4r(T)$

Pf of claim:

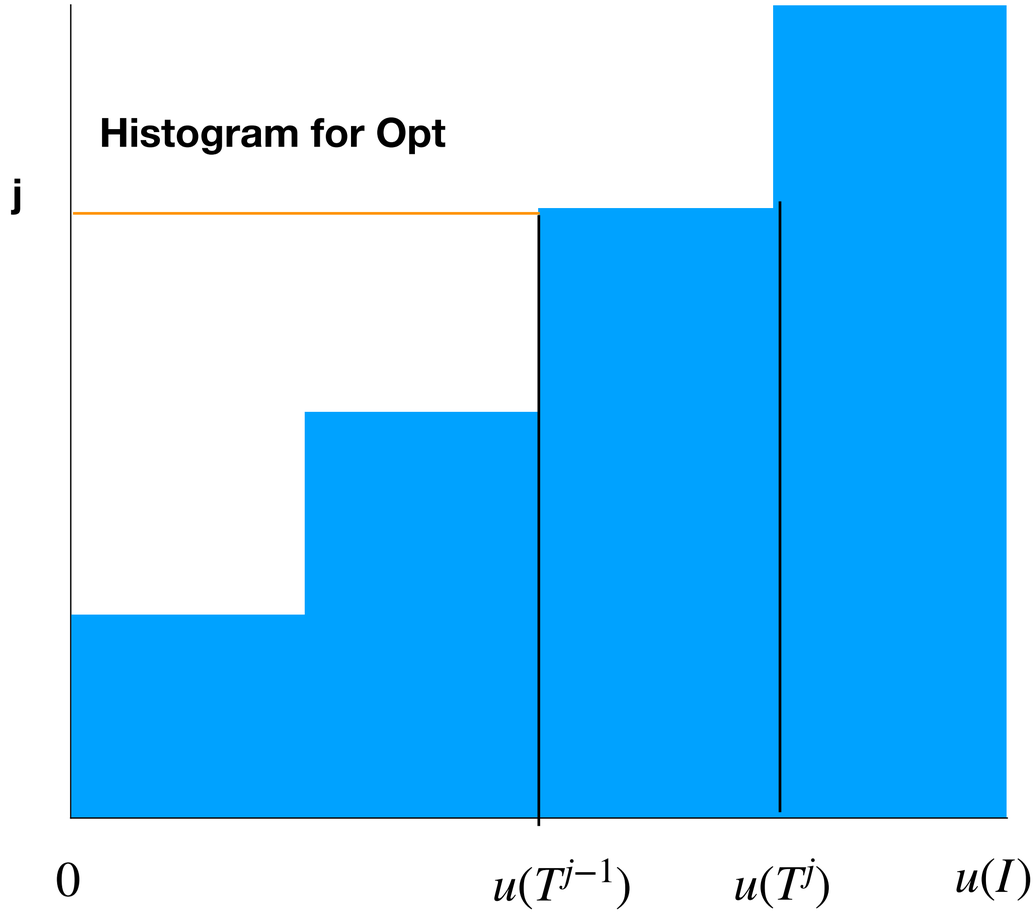
Consider a histogram for T.

Horizontal axis: labeled from 0 to $u(I)$

Bar corresponding to T_j goes from

$$u(T^{j-1}) \text{ to } u(T^j)$$

of height j



Area under histogram is $r(T) = \sum_{j=1}^{m} j \times (u(T^j) - u(T^{j-1}))$

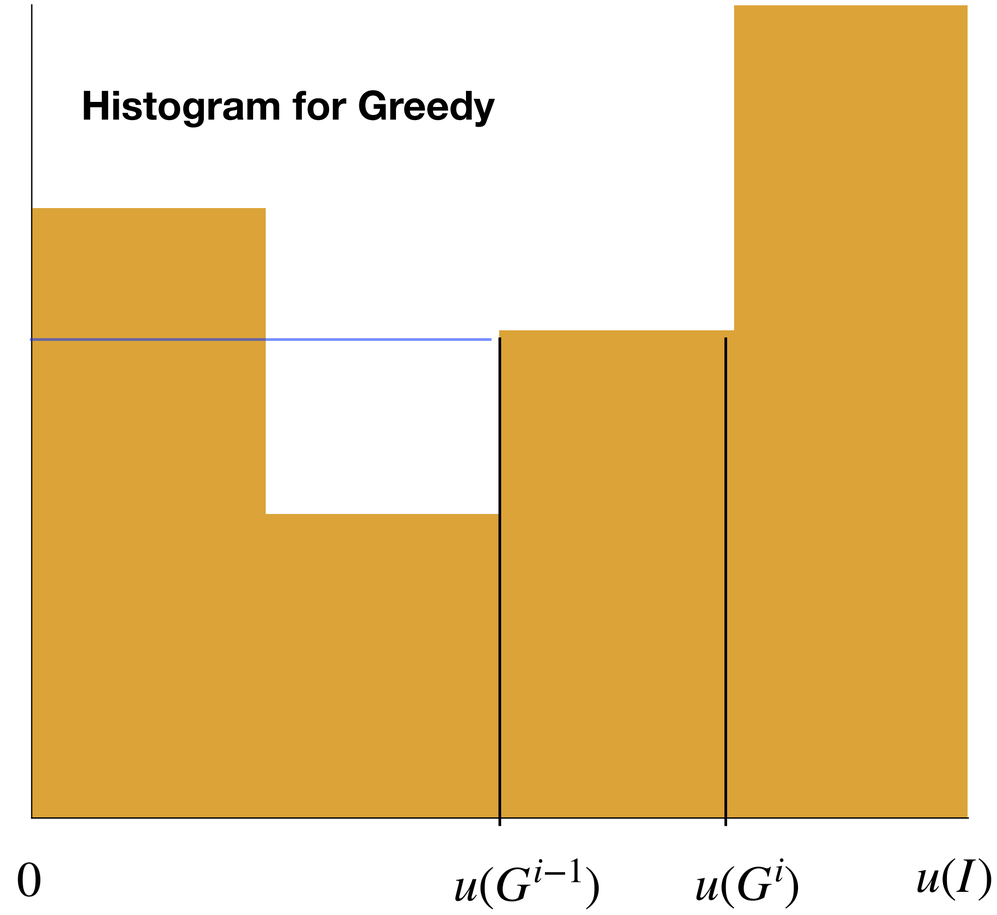
Consider a histogram for G.

Horizontal axis: labeled from 0 to $u(N)$

Bar corresponding to G_i goes from

$$u(G^{i-1}) \text{ to } u(G^i)$$

of height $(u(I) - u(G^{i-1})) \times \dfrac{1}{u(G^i) - u(G^{i-1})}$



Area under histogram is $\sum_{i=1}^{m} 1 \times (u(I) - u(G^{i-1})) = r(G)$

Consider histogram for G.

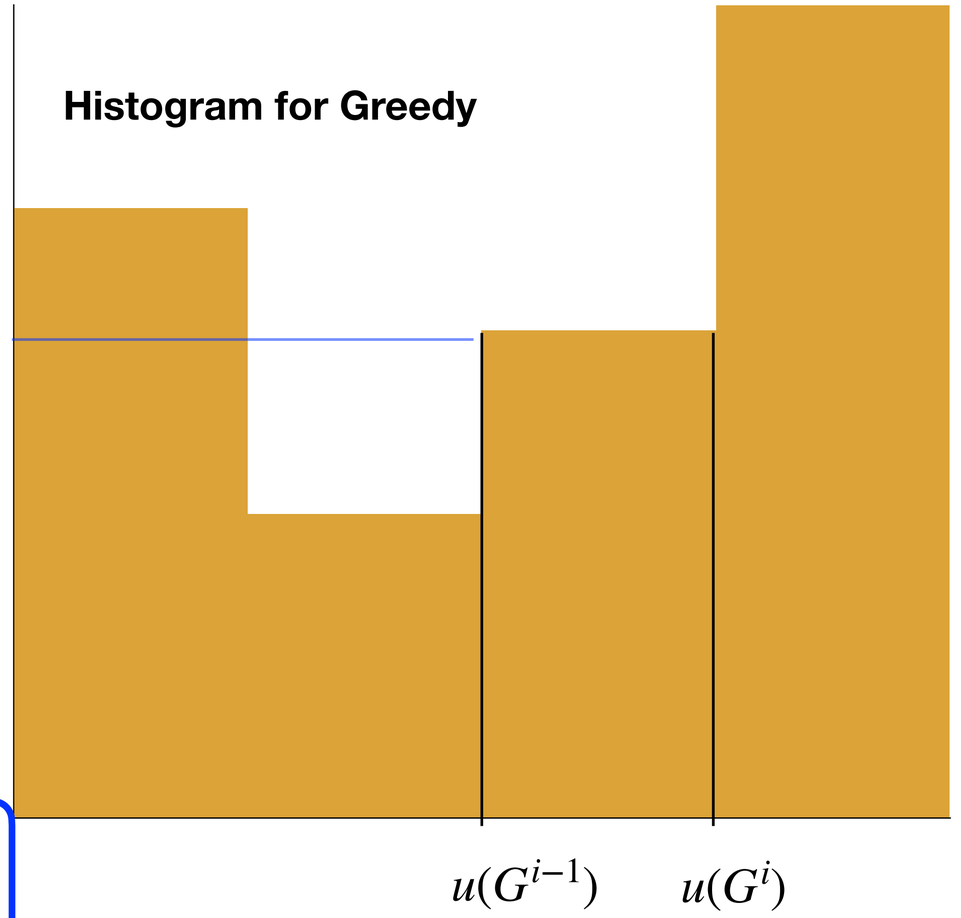
Horizontal axis: labeled from 0 to $u(N)$

Bar corresponding to G_i goes from

$$u(G^{i-1}) \text{ to } u(G^i)$$

and height $(u(I) - u(G^{i-1})) * \dfrac{1}{u(G^i) - u(G^{i-1})}$

Area under histogram is $\sum_i 1 \times (u(I) - u(G^{i-1})) = r(G)$



Inverse of utility gained by Greedy in step i

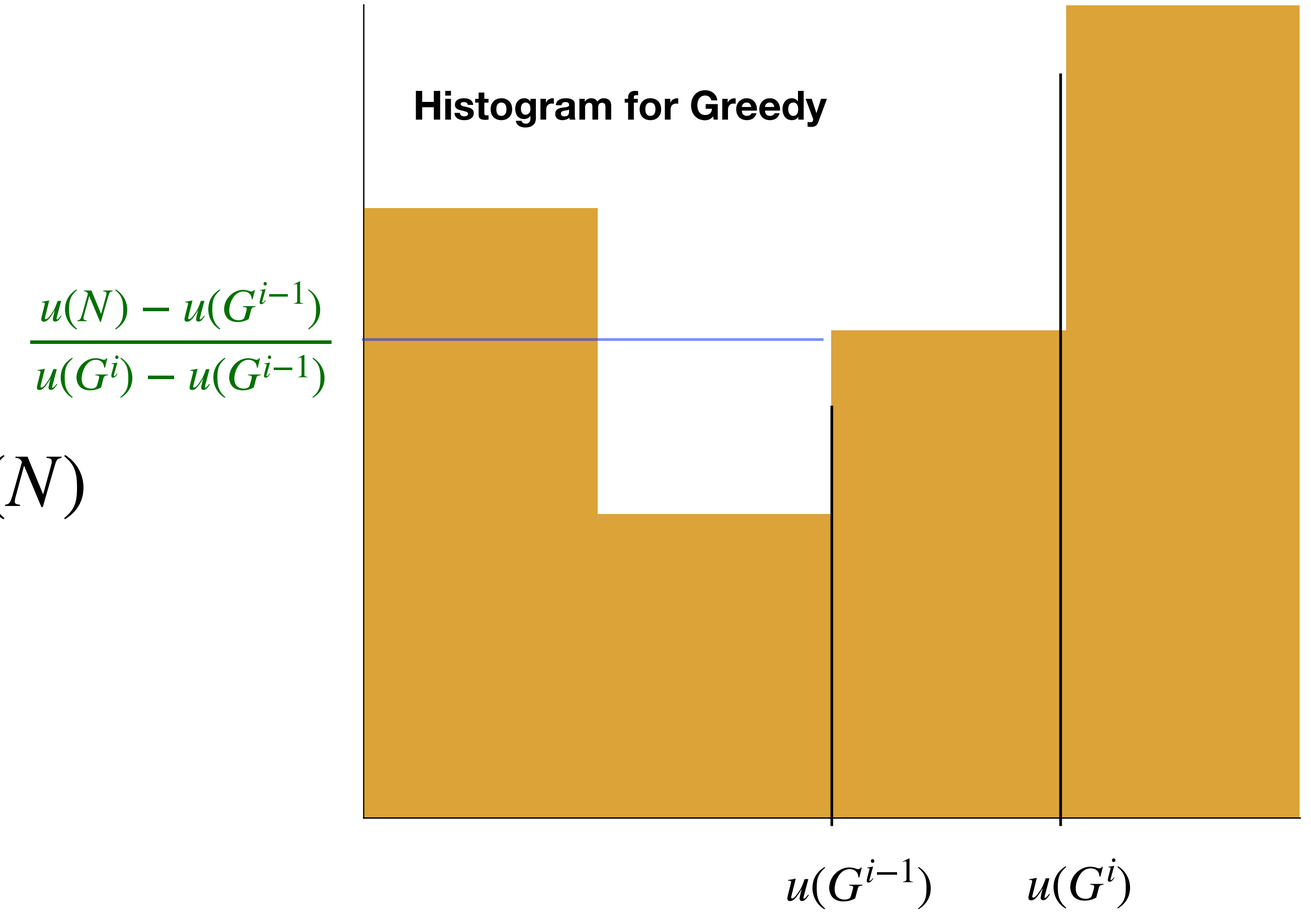
Consider histogram for G.

Horizontal axis: labeled from 0 to $u(N)$

Bar corresponding to G_i goes from

$$u(G^{i-1}) \text{ to } u(G^i)$$

of height $\dfrac{u(N) - u(G^{i-1})}{u(G^i) - u(G^{i-1})}$



Area under histogram is $\sum_i 1 \times (u(N) - u(G^{i-1})) = r(G)$

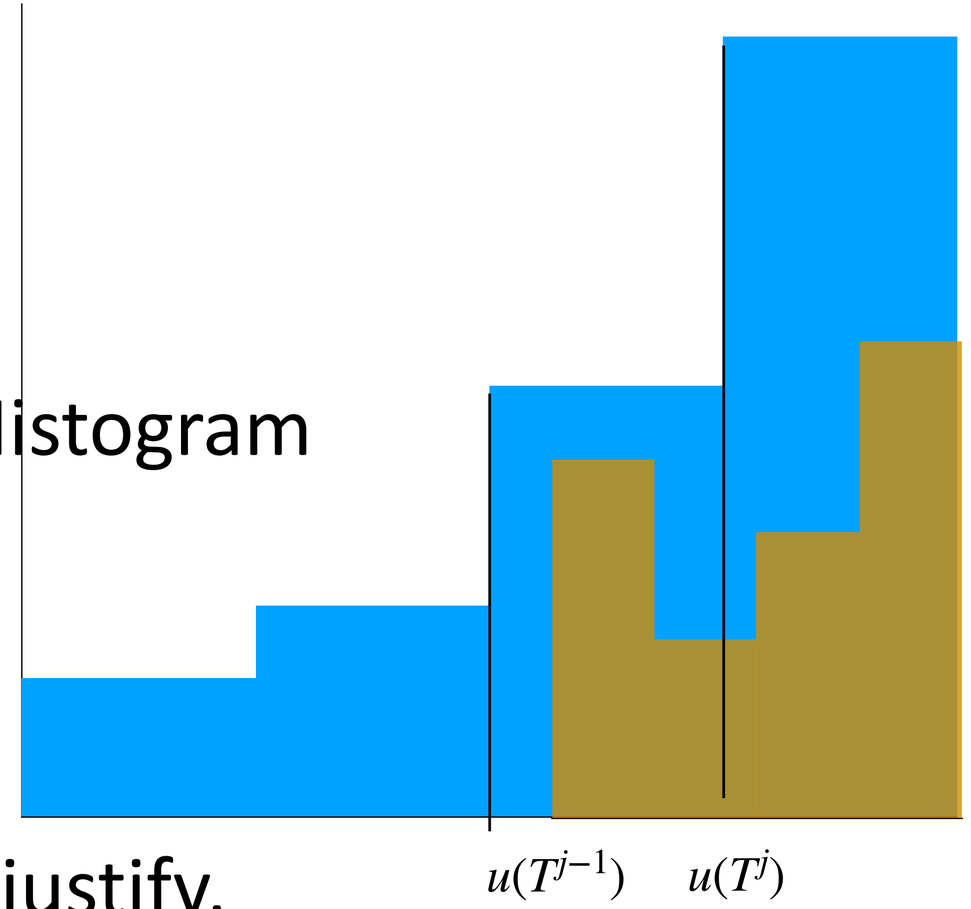
Claim: $r(G) \leq 4r(T)$

Pf of claim (continued):

We'll show if shrink Greedy (brown) Histogram by factor of 2 in both horizontal and

vertical directions
(decreasing its area by 4),

overlay it on optimal histogram, right justify,
then it "fits" into Optimal (blue) Histogram



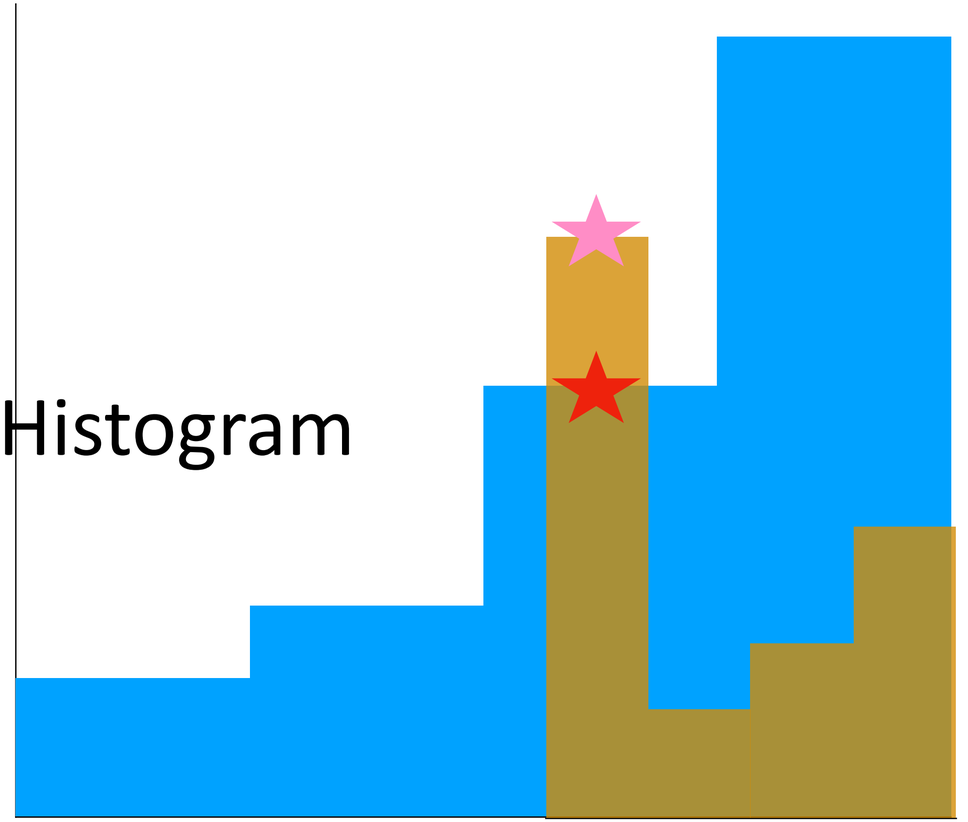
Claim: $r(G) \leq 4r(T)$

Pf of claim (continued):

Suppose not. Suppose shrink Greedy Histogram by factor of 2 in horizontal and

vertical directions, overlay it on optimal histogram, right justify,

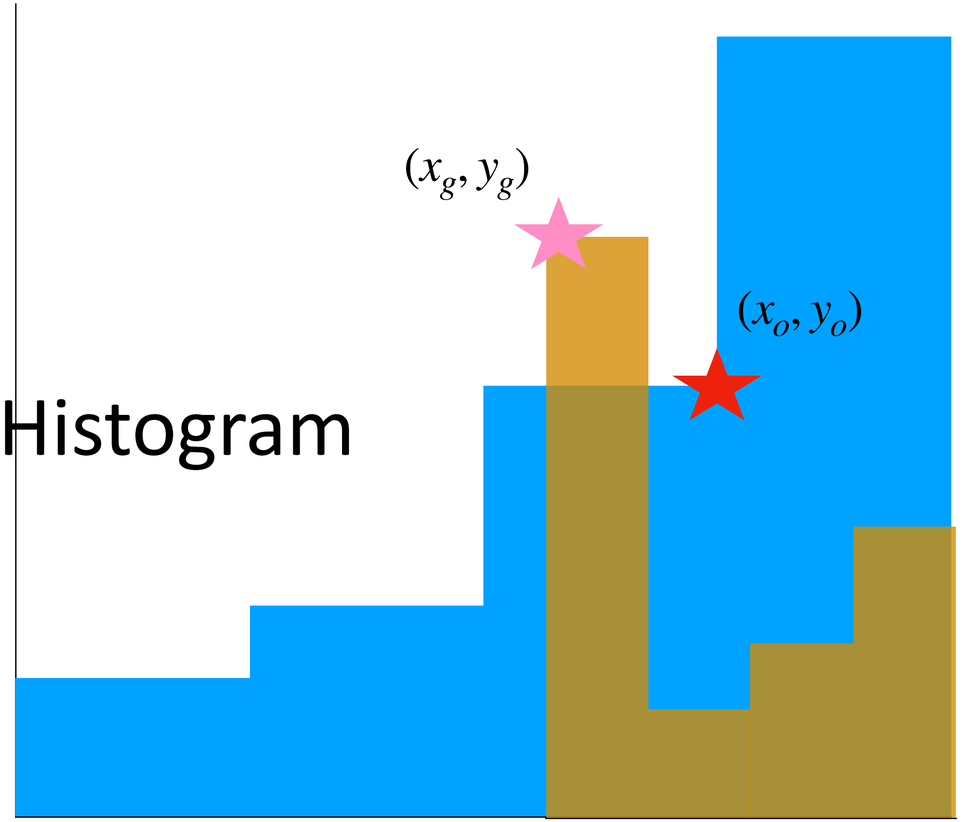
and it *doesn't* fit into Optimal Histogram



Claim: $r(G) \leq 4r(T)$

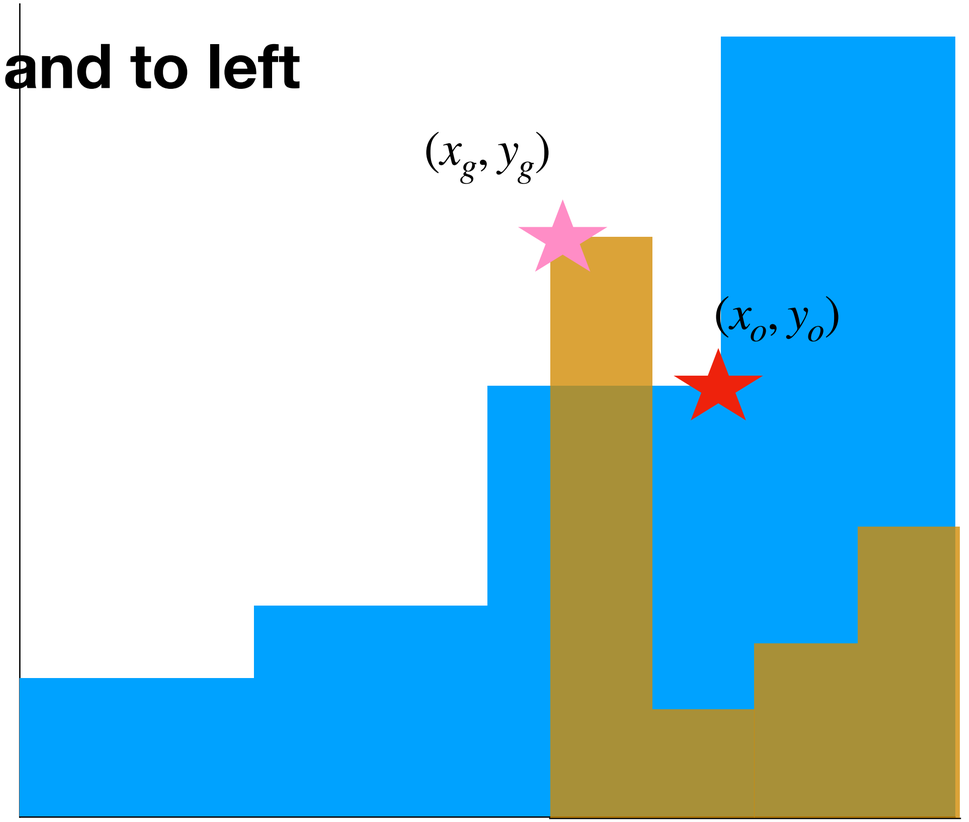
Pf of claim (continued):

Suppose not. Suppose shrink Greedy Histogram by factor of 2 in horizontal and vertical directions, overlay it on optimal histogram, right justify, and it *doesn't* fit into Optimal Histogram



Then a vertical bar of the shrunken Greedy (brown) histogram rises above the Optimal (blue) Histogram at some point. Top left corner of the brown bar is above and to left of top right corner of lower blue bar.

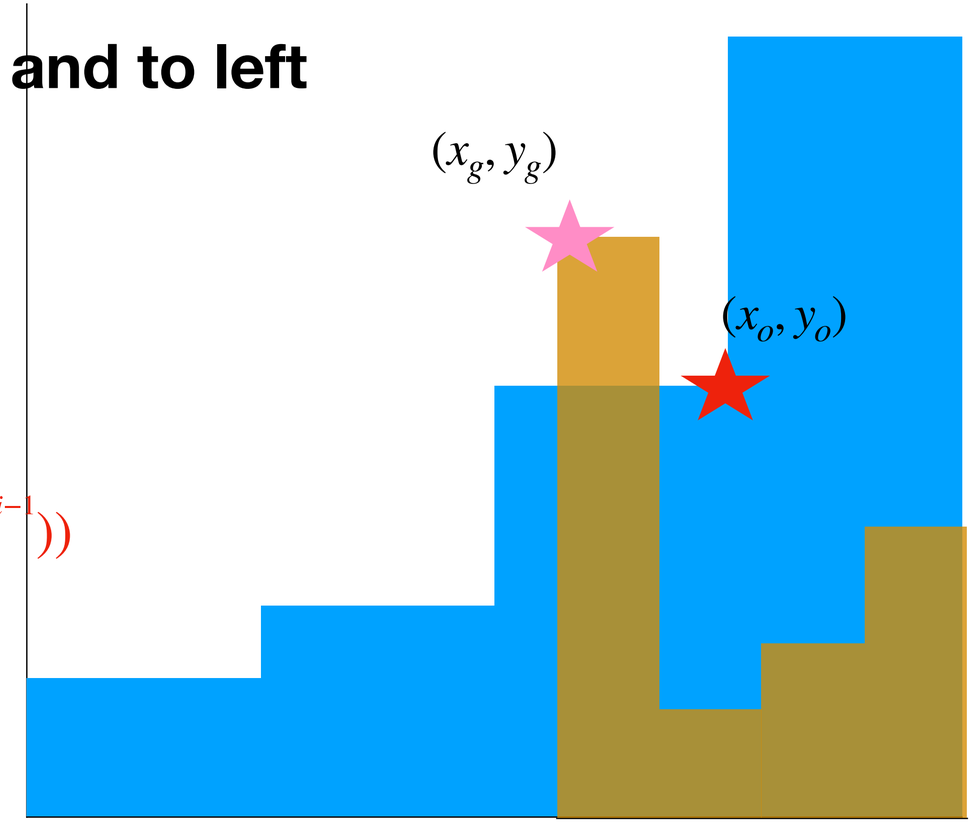
Then top left corner of the brown bar is above and to left of top right corner of lower blue bar.



$$\frac{1}{2} * \frac{u(I) - u(G^{i-1})}{u(G^i) - u(G^{i-1})} > j \qquad y_g > y_o$$

$$u(I) - u(T_j) \geq \frac{1}{2}(u(I) - u(G^{i-1})) \qquad x_g \leq x_o \text{ or equivalently } u(I) - x_o \geq \frac{1}{2}(u(I) - x_g)$$

Then top left corner of the brown bar is above and to left of top right corner of lower blue bar.



But recall Lemma:

If $\frac{u(G^i) - u(G^{i-1})}{u(N) - u(G^{i-1})} < \frac{1}{2j}$ then $u(I) - u(T^j) \geq \frac{1}{2}(u(I) - u(G^{i-1}))$

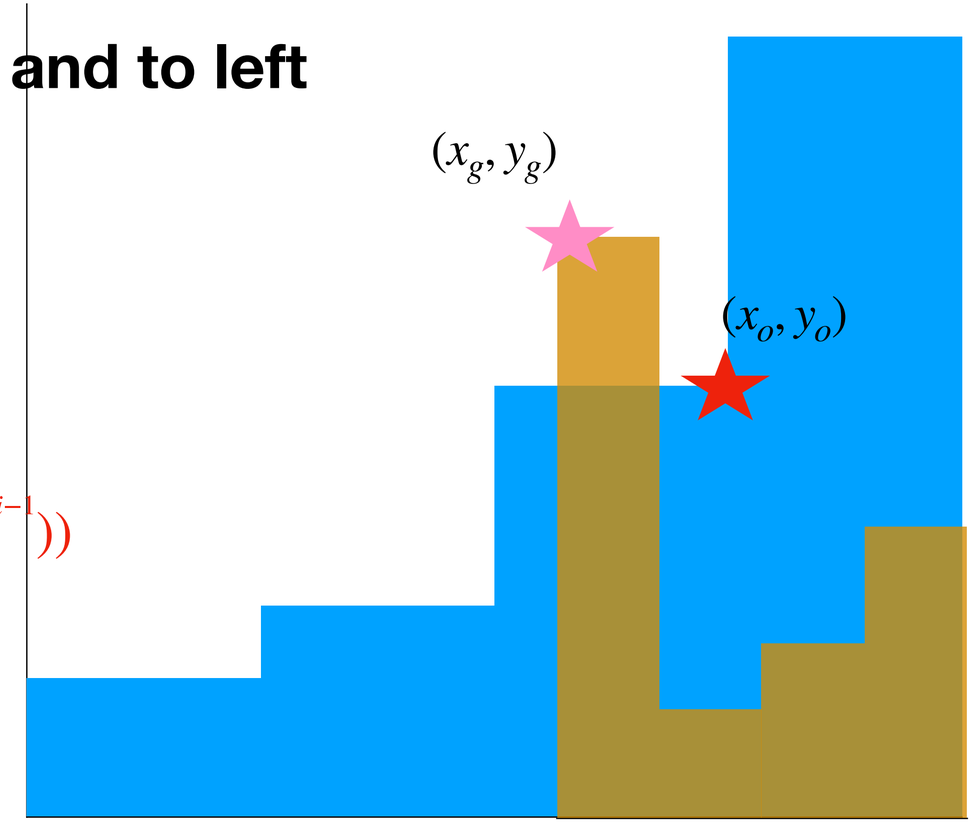
$$\frac{1}{2} * \frac{u(I) - u(G^{i-1})}{u(G^i) - u(G^{i-1})} > j$$

$$y_g > y_o$$

$$u(I) - u(T_j) \geq \frac{1}{2}(u(I) - u(G^{i-1}))$$

$x_g \leq x_o$ or equivalently $u(I) - x_o \geq \frac{1}{2}(u(I) - x_g)$

Then top left corner of the brown bar is above and to left of top right corner of lower blue bar.



But recall Lemma:

If $\frac{u(G^i) - u(G^{i-1})}{u(N) - u(G^{i-1})} < \frac{1}{2j}$ then $u(I) - u(T^j) \geq \frac{1}{2}(u(I) - u(G^{i-1}))$

CONTRADICTION!

$$\frac{1}{2} * \frac{u(I) - u(G^{i-1})}{u(G^i) - u(G^{i-1})} > j$$

$$y_g > y_o$$

$$u(I) - u(T_j) \geq \frac{1}{2}(u(I) - u(G^{i-1}))$$

$x_g \leq x_o$ or equivalently $u(I) - x_o \geq \frac{1}{2}(u(I) - x_g)$

- QED, proved that Greedy Algorithm produces a solution with sum of covering time $\leq 4 \times$ OPT
- NP-hard to achieve $(4 - \epsilon) \times$ OPT, for every $\epsilon > 0$
- Weighted version of Min-Sum Set Cover:
  - each subset S_j of elements of the universe has weight w_j , find π minimizing sum of weighted covering times:

$$\sum_{i=1}^{m} w_{\pi[i]} \times (u(\pi^i) - u(\pi^{i-1}))$$

  - weighted greedy algorithm (bang-for-the-buck) produces a solution with value $\leq 4 \times OPT$
  - also called “pipelined set cover”

More general functions u

- Min-Sum Submodular Cover
  - Function u can be arbitrary monotone submodular utility function $u : 2^I \rightarrow \mathscr{R}^{\geq 0}$
  - analogous weighted greedy algorithm also produces a solution with value $\leq 4 \times$ OPT
- Min-Sum Ordering Problems, Minimum Linear Ordering Problems
  - Very general class of problems that includes all of the above min-sum cover generalizations, with wide range of functions u and weight (cost) functions
  - If u is monotone, get a $4 \times$ OPT approximation by using a generalization of the greedy algorithm using greedy rule that instead of choosing a single item in each greedy step, chooses subset of elements of max “density”

Other min-sum set cover variants

- Minimum-Latency Set Cover
  - Change definition of covering time of element $e \in \mathcal{U}$ in a permutation π of I. It's the earliest step in the permutation at which ALL j such that $e \in S_j$ have appeared.
  - A number of poly-time approximation algorithms, approximation is $2 \times$ OPT
  - Studied in the scheduling literature
- Generalized Min-Sum Set Cover
  - Have a *covering requirement* $k(e)$ for each $e \in \mathcal{U}$. Change definition of covering time of element e in a permutation π of N. It's the earliest step in the permutation at which $k(e)$ of the j such that $e \in S_j$ have appeared.
  - (no weights)
  - Generalizes Min-Sum Set Cover and Min-Latency Submodular Cover
  - Current best approximation achieved by poly-time algorithm is 4.642 $\times$ OPT [Bansal et al. 2023]

- Many others…

Questions?